Week 6: DDL, DML and Python with SQLite

DDL defines structure, DML changes data

```
CREATE TABLE member (
  member_id INTEGER PRIMARY KEY,
  name   TEXT NOT NULL CHECK (length(name) >= 2),
  email  TEXT UNIQUE,
  joined TEXT NOT NULL DEFAULT (date('now'))
);
ALTER TABLE member ADD COLUMN phone TEXT;
DROP TABLE member;          -- deletes the table!
```

Constraint	Rule
`PRIMARY KEY`	Unique, identifies the row
`NOT NULL`	A value is required
`UNIQUE`	No two rows the same
`CHECK (cond)`	Condition must be true
`DEFAULT x`	Used when no value is given
`FOREIGN KEY (c) REFERENCES t (c)`	Must match a row in `t`

> Turn foreign keys on in **every** session or connection: `PRAGMA foreign_keys = ON;`. SQLite does not enforce them otherwise.

DML: INSERT, UPDATE, DELETE

```
INSERT INTO member (name, email)
VALUES ('Mei Lin', 'mei@example.com');
UPDATE member SET email = 'new@example.com'
WHERE member_id = 1;
DELETE FROM member WHERE member_id = 1;
```

Always write the `WHERE` on UPDATE and DELETE; without it every row changes. Test it as a `SELECT` first. Delete **child** rows before the parent.

Python and SQLite

```
import sqlite3
con = sqlite3.connect("my-work/library.db")
con.row_factory = sqlite3.Row   # row["title"]
con.execute("PRAGMA foreign_keys = ON")

cur = con.execute(
    "SELECT * FROM book WHERE author = ?", (author,))
rows = cur.fetchall()           # list of rows
one  = cur.fetchone()           # one row or None

with con:                       # commit, or roll back
    con.execute("INSERT INTO book (title, author) "
                "VALUES (?, ?)", (title, author))
con.close()
```

- `?` placeholders take **values** (never table or column names).
- A one-value tuple needs a comma: `(author,)`.
- `cur.lastrowid` = the new id. `cur.rowcount` = rows changed.
- `con.commit()` saves; without it, changes are lost. `with con:` commits or rolls back for you.

SQL injection

```
# DANGEROUS: user text becomes part of the SQL
con.execute(f"SELECT * FROM users WHERE name='{n}'")
#  n = ' OR '1'='1      -> every user returned
#  n = x' --   -> the rest becomes a comment

# SAFE: the value is passed separately
con.execute(
    "SELECT * FROM users WHERE name = ?", (n,))
```

LIKE with a placeholder: `WHERE name LIKE ?` with `(f"%{term}%",)`.

Transactions

A group of changes that must **all** happen or **none**. If an error occurs inside `with con:`, every change in the block is undone.

Top mistakes

- f-strings or `+` in SQL. Use `?`.
- Forgetting to commit.
- Forgetting `PRAGMA foreign_keys = ON`.
- `UPDATE`/`DELETE` with no `WHERE`.